Quantifying Attention Flow in Transformers

Samira Abnar, Willem Zuidema
University of Amsterdam

Simon Kraft
University of Northern British Columbia

March 18, 2026

Transformer Models

Basics

Proposed by famous Google paper *Attention Is All You Need* [2].

- Numerous applications in NLP, Computer Vision, ...
- Process sequence of input **tokens**
- Stack of identical layers
- Each applying **Self-Attention**

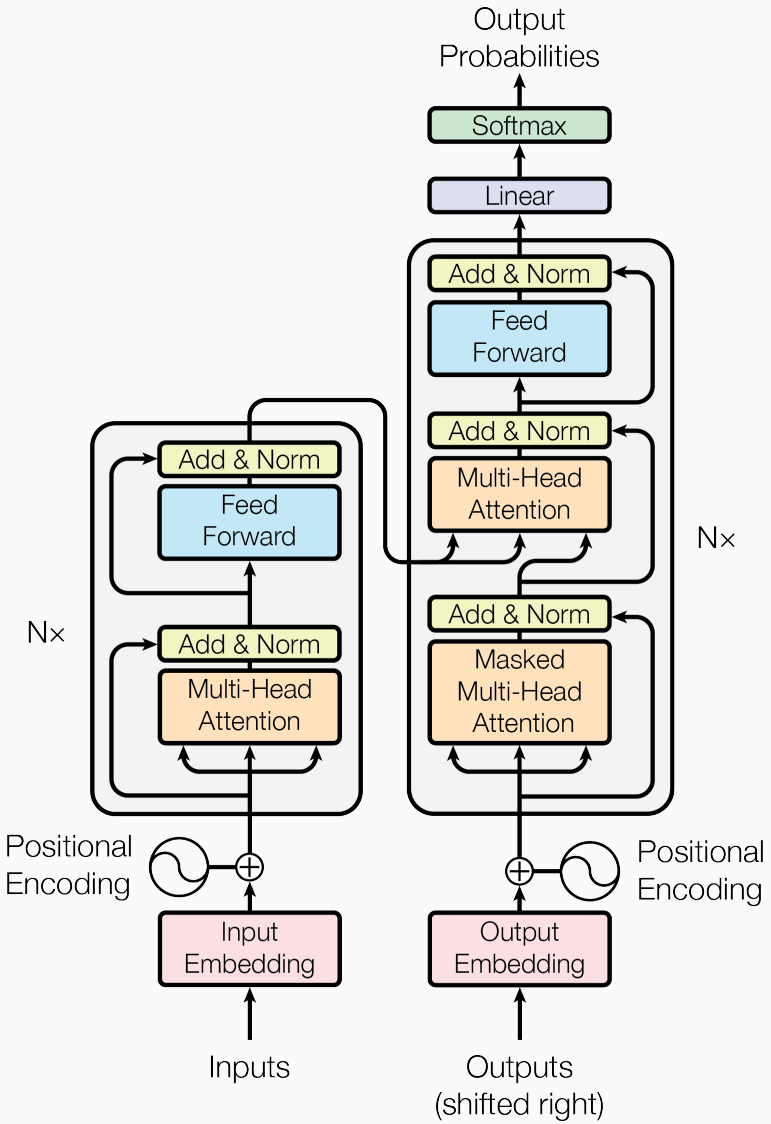


Figure 1: Transformer Model. Taken from [2].

Self-Attention

Self-Attention

- For sequence of input tokens $X = (x_1, \ldots, x_n) \in \mathbb{R}^{n \times d}$ compute self-attention with:

Self-Attention

- For sequence of input tokens $X = (x_1, \ldots, x_n) \in \mathbb{R}^{n \times d}$ compute self-attention with:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

Self-Attention

- For sequence of input tokens $X = (x_1, \ldots, x_n) \in \mathbb{R}^{n \times d}$ compute self-attention with:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

where the matrices Q, K, and V are different linear projections of the input tokens X

Self-Attention

- For sequence of input tokens $X = (x_1, \ldots, x_n) \in \mathbb{R}^{n \times d}$ compute self-attention with:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

  where the matrices Q, K, and V are different linear projections of the input tokens X

- $\text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)$ regarded as **attention weights**

Self-Attention

- For sequence of input tokens $X = (x_1, \ldots, x_n) \in \mathbb{R}^{n \times d}$ compute self-attention with:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

  where the matrices Q, K, and V are different linear projections of the input tokens X

- $\text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)$ regarded as **attention weights**

Result

After this operation, each token captures **contextual information from all other tokens**.

Problem Statement

Problem Statement

Token Processing

Problem Statement

Token Processing

- Self-attention operation is executed in **each layer**

Problem Statement

Token Processing

- Self-attention operation is executed in **each layer**
- Across layers, **information** from different tokens gets increasingly **mixed**

Problem Statement

Token Processing

- Self-attention operation is executed in **each layer**
- Across layers, **information** from different tokens gets increasingly **mixed**

Research Question

Problem Statement

Token Processing

- Self-attention operation is executed in **each layer**
- Across layers, **information** from different tokens gets increasingly **mixed**

Research Question

How can we quantify this information flow across multiple layers?

Idea

Idea

Model information flow in the network with a **Directed Acyclic Graph (DAG)**:

Idea

Model information flow in the network with a **Directed Acyclic Graph (DAG)**:

- **Nodes**: Input tokens & hidden embeddings

Idea

Model information flow in the network with a **Directed Acyclic Graph (DAG)**:

- **Nodes**: Input tokens & hidden embeddings
- **Edges**: Attention weights from nodes in each layer to those in previous layer

Idea

Model information flow in the network with a **Directed Acyclic Graph (DAG)**:

- **Nodes**: Input tokens & hidden embeddings
- **Edges**: Attention weights from nodes in each layer to those in previous layer

Verb Number Prediction Task

Predicting singularity or plurality of a verb of a sentence, when the input is the sentence up to the verb position

Attention Flow DAG

Input: <cls> <bos> the key to the cabinets <verb>

Attention Flow DAG

Input: <cls> <bos> the key to the cabinets <verb>

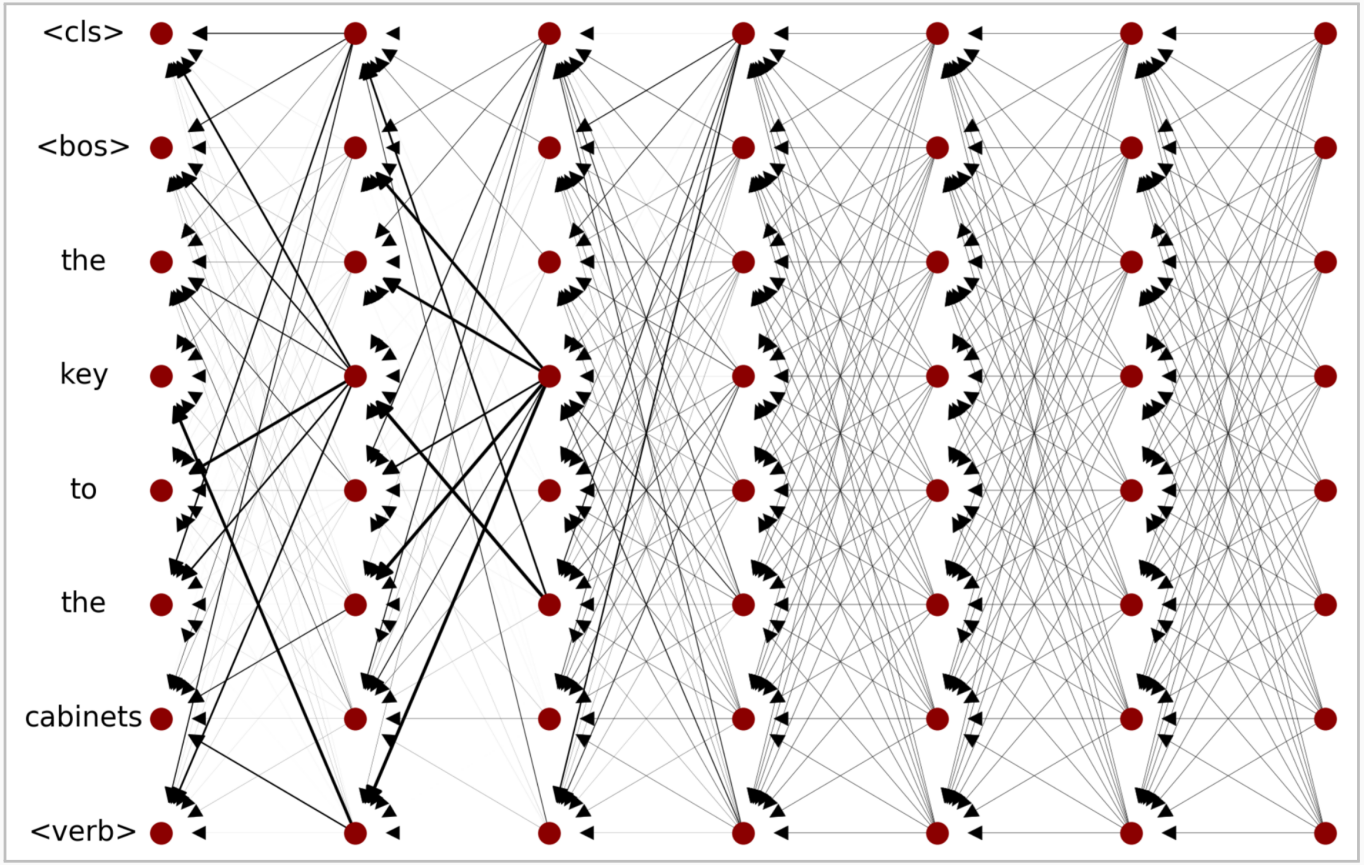


Figure 2: Attention Flow Graph of input sentence.

Raw Attention Visualized

Input: <cls> <bos> the key to the cabinets <verb>

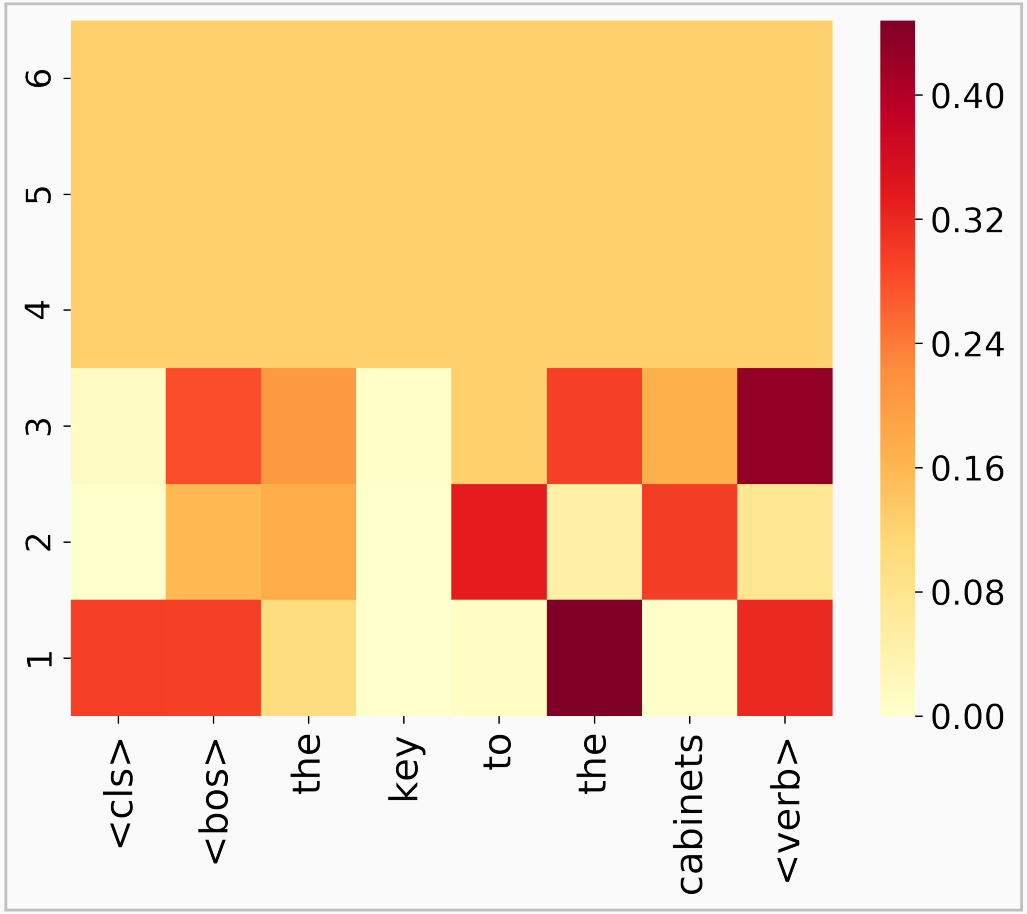


Figure 3: Raw Attention map for the <cls> token at different layers.

Attention Rollout

Methods

Attention Rollout

- Recursively multiply **attention weight matrices** across transformer layers:

Methods

Attention Rollout

- Recursively multiply **attention weight matrices** across transformer layers:

$$\tilde{A}^{(l)} = \left(A^{(l)} + \mathbb{I}\right) \cdot \tilde{A}^{(l-1)}, \quad \tilde{A}^{(0)} = \mathbb{I}$$

Methods

Attention Rollout

- Recursively multiply **attention weight matrices** across transformer layers:

$$\tilde{A}^{(l)} = \left(A^{(l)} + \mathbb{I}\right) \cdot \tilde{A}^{(l-1)}, \quad \tilde{A}^{(0)} = \mathbb{I}$$

where $A^{(l)} =$ are the attention weights from layer l

Attention Rollout

- Recursively multiply **attention weight matrices** across transformer layers:

$$\tilde{A}^{(l)} = \left(A^{(l)} + \mathbb{I}\right) \cdot \tilde{A}^{(l-1)}, \quad \tilde{A}^{(0)} = \mathbb{I}$$

  where $A^{(l)} =$ are the attention weights from layer l

- Whole operation in $\mathcal{O}(d \times n^2)$

Methods

Attention Rollout

- Recursively multiply **attention weight matrices** across transformer layers:

$$\tilde{A}^{(l)} = \left(A^{(l)} + \mathbb{I}\right) \cdot \tilde{A}^{(l-1)}, \quad \tilde{A}^{(0)} = \mathbb{I}$$

  where $A^{(l)} =$ are the attention weights from layer l

- Whole operation in $\mathcal{O}(d \times n^2)$

Notation: d is model depth, n is sequence length.

Attention Flow

Methods

Attention Flow

- Attention graph as a **flow network**

Methods

Attention Flow

- Attention graph as a **flow network**
- Capacities of edges are attention weights

Methods

Attention Flow

- Attention graph as a **flow network**
- Capacities of edges are attention weights
- **Maximum flow algorithm** to compute attention flow from **one** node in output layer to **one** node in input layer

Attention Flow

- Attention graph as a **flow network**
- Capacities of edges are attention weights
- **Maximum flow algorithm** to compute attention flow from **one** node in output layer to **one** node in input layer
- One computation in $\mathcal{O}(d^2 \times n^4)$

Notation: d is model depth, n is sequence length.

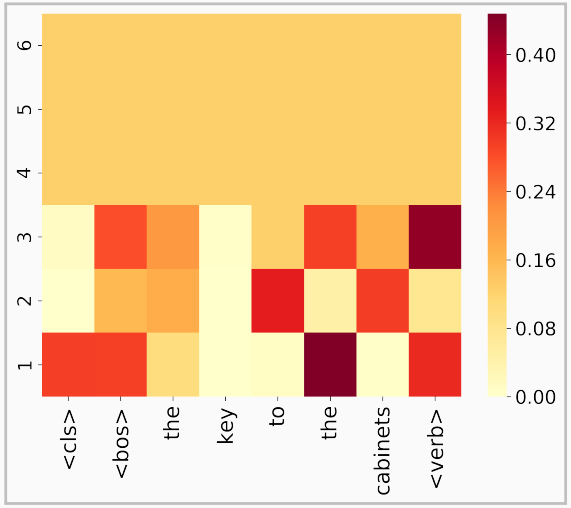
Results

Input: <cls> <bos> the key to the cabinets <verb>

Results

Input: <cls> <bos> the key to the cabinets <verb>

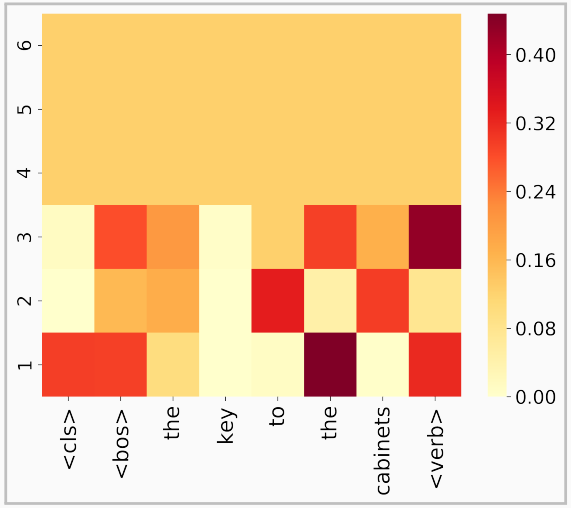
Raw



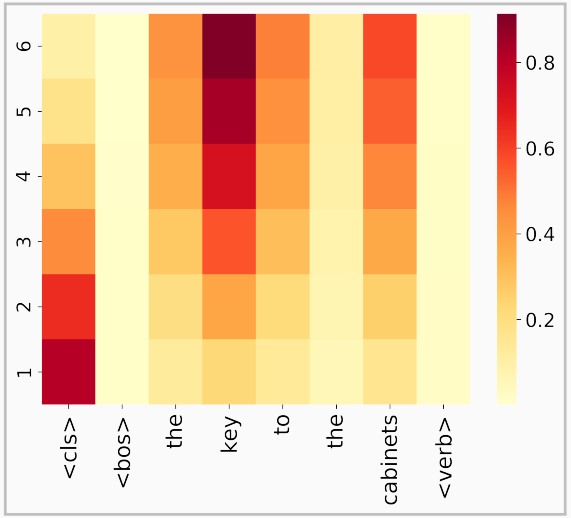
Results

Input: <cls> <bos> the key to the cabinets <verb>

Raw



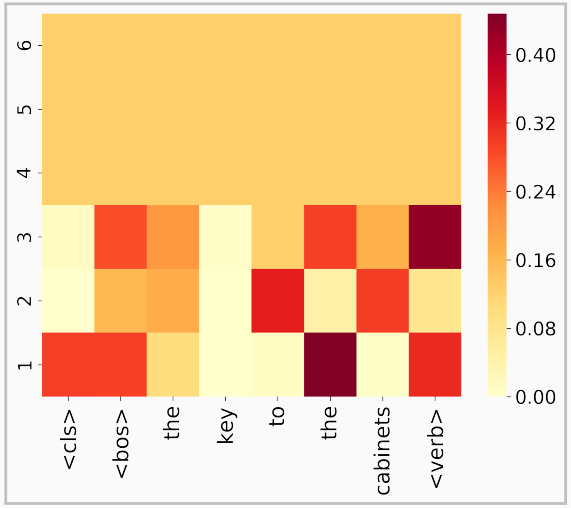
Rollout



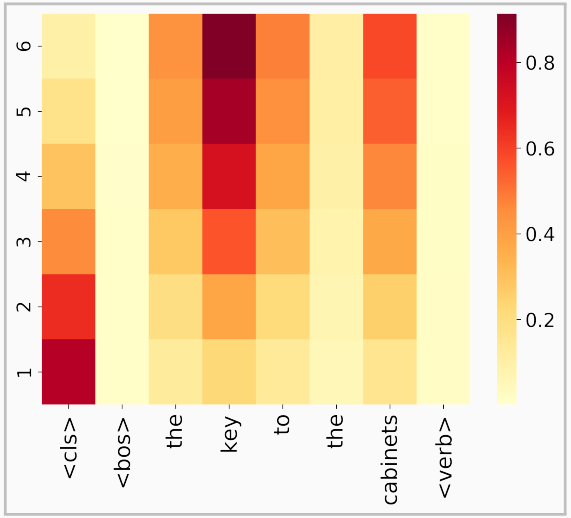
Results

Input: <cls> <bos> the key to the cabinets <verb>

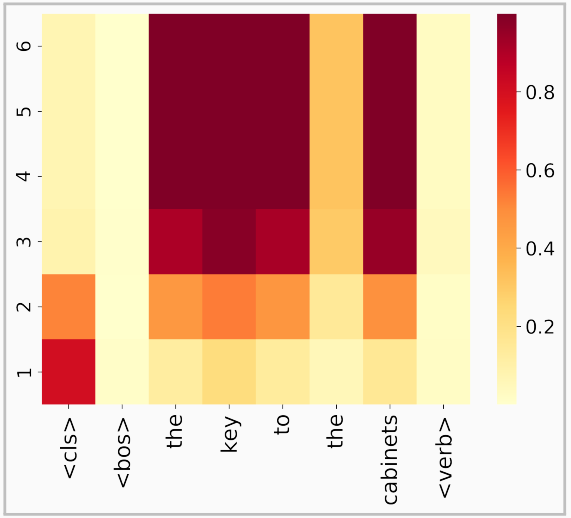
Raw



Rollout



Flow



Results

Input: <cls> <bos> the key to the cabinets <verb>

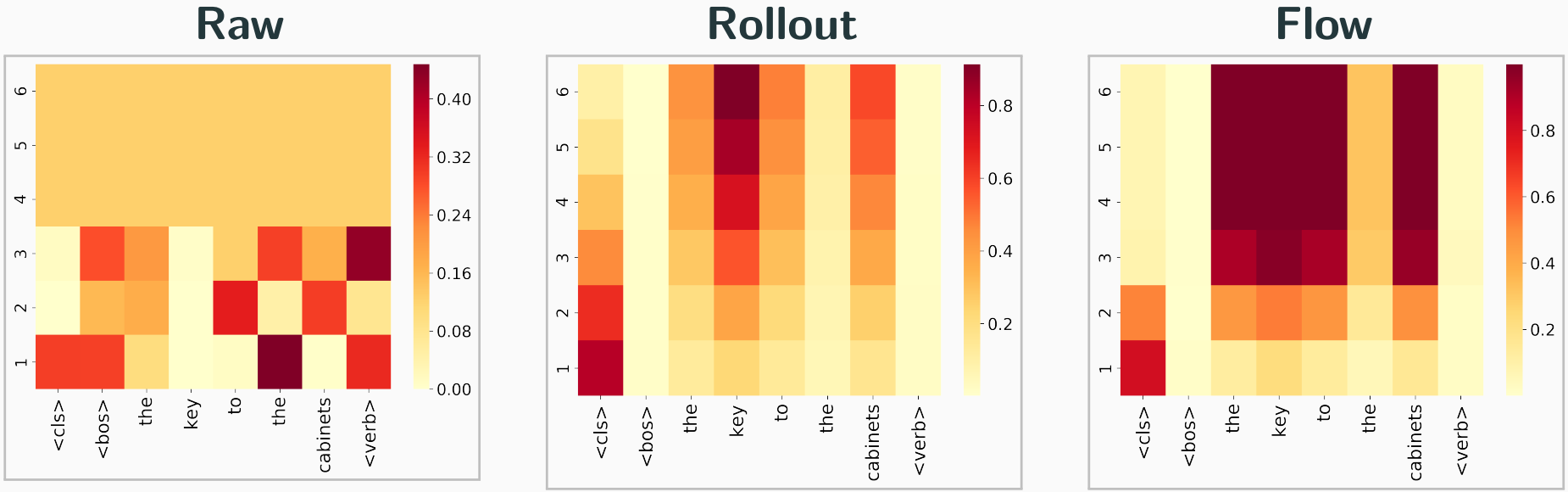


Figure 4: Comparison of attention maps for the <cls> token at different layers.

Results

Input: <cls> <bos> the key to the cabinets <verb>

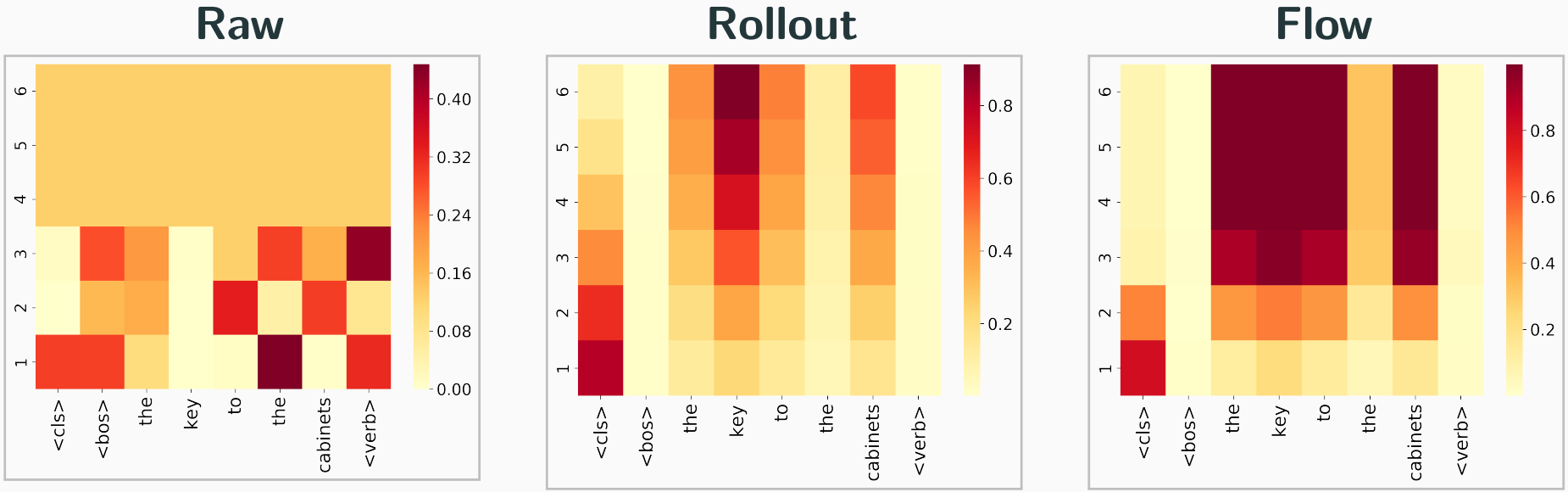


Figure 4: Comparison of attention maps for the <cls> token at different layers.

Correctly classified?

Results

Input: <cls> <bos> the key to the cabinets <verb>

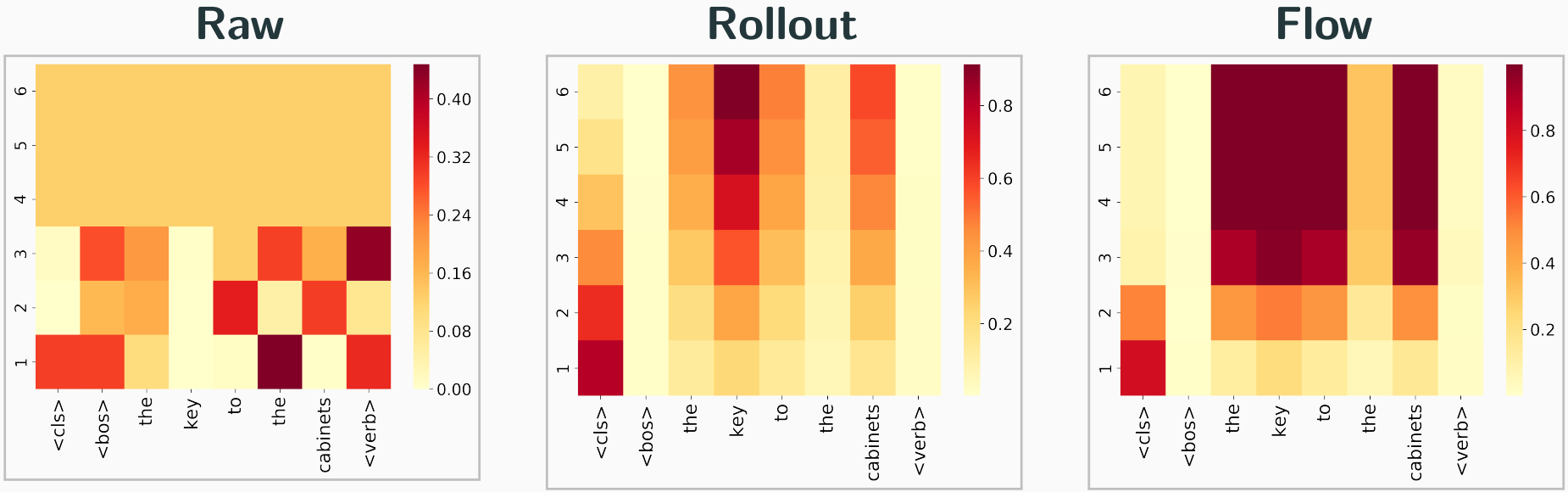


Figure 4: Comparison of attention maps for the <cls> token at different layers.

Correctly classified? ✓

Results

Input: <cls> <bos> here the NNS differ in that the female <verb>

Results

Input: <cls> <bos> here the NNS differ in that the female <verb>

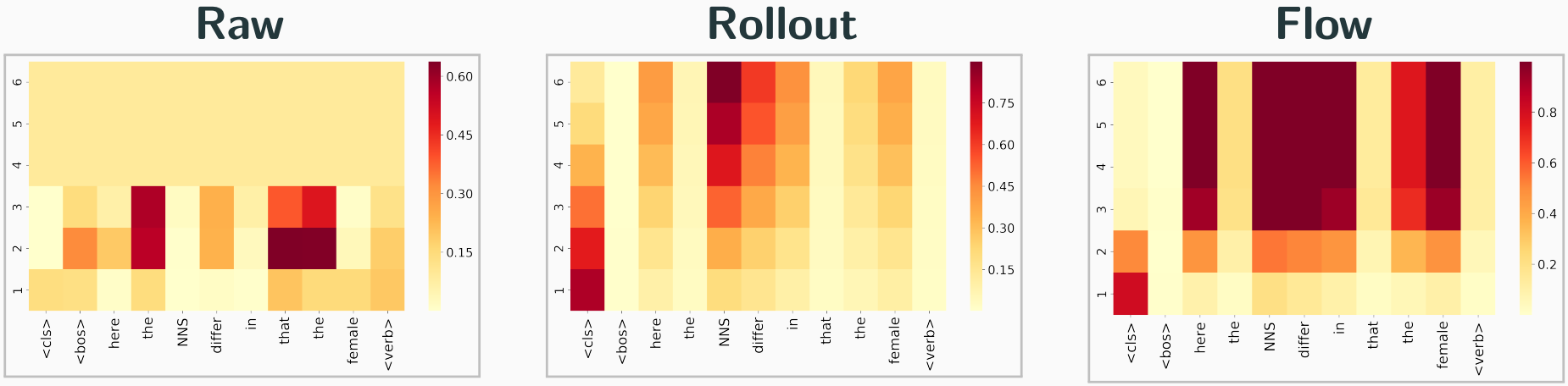


Figure 5: Comparison of attention maps for the <cls> token at different layers.

Results

Input: <cls> <bos> here the NNS differ in that the female <verb>

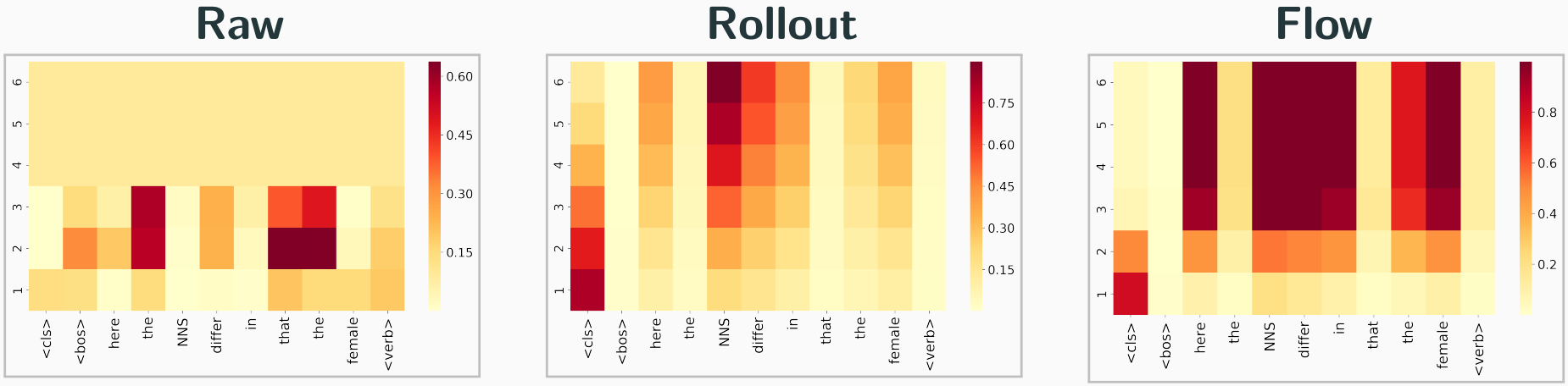


Figure 5: Comparison of attention maps for the <cls> token at different layers.

Correctly classified?

Results

Input: <cls> <bos> here the NNS differ in that the female <verb>

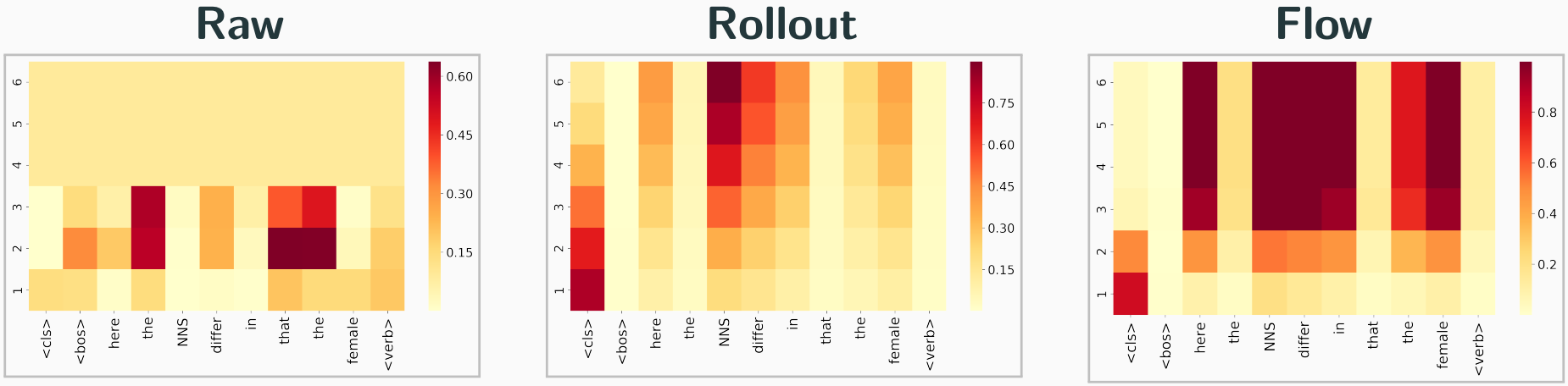


Figure 5: Comparison of attention maps for the <cls> token at different layers.

Correctly classified? ✗

Conclusion

Attention Rollout & Attention Flow

Conclusion

Attention Rollout & Attention Flow

- Methods for explaining attention in transformer models

Conclusion

Attention Rollout & Attention Flow

- Methods for explaining attention in transformer models
- Architecture agnostic

Conclusion

Attention Rollout & Attention Flow

- Methods for explaining attention in transformer models
- Architecture agnostic
- Provide insights into correct classifications and misclassifications

Conclusion

Attention Rollout & Attention Flow

- Methods for explaining attention in transformer models
- Architecture agnostic
- Provide insights into correct classifications and misclassifications
- Attention Rollout computationally more feasible and more accurate results

Thank you for your attention!

Any questions?

References

[1] S. Abnar and W. Zuidema, **"Quantifying attention flow in transformers,"** in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky et al., Eds., Online: Association for Computational Linguistics, Jul. 2020, pp. 4190–4197. DOI: 10.18653/v1/2020.acl-main.385. [Online]. Available: https://aclanthology.org/2020.acl-main.385/.

[2] A. Vaswani et al., **Attention is all you need,** 2023. arXiv: 1706.03762 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1706.03762.

[3] A. Radford et al., **"Language models are unsupervised multitask learners,"**, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:160025533.

[4] T. Linzen, E. Dupoux, and Y. Goldberg, **Assessing the ability of lstms to learn syntax-sensitive dependencies,**

Setup

Model

- Transformer encoder model with 6 layers
- GPT-2 blocks from [3]
- Embedding of `<cls>` as input to classifier
- 96% final accuracy on verb number prediction task

Dataset

- Public subject-verb agreement dataset [4]